Arquitectura de Observabilidad con OpenTelemetry

Carlos Arturo Salazar Castaneda

Juan Sebastian Buitrago Romero

Juan Jose Velez Alvarez

Jhonatan Winston Sotelo

Santiago Eduardo Muñoz Castillo

12 de octubre de 2026

Arquitectura de Observabilidad con OpenTelemetry

Instrumentación Python, OTel Collector y análisis de overhead

Resumen ejecutivo

La solución separa la generación de telemetría de los backends de observabilidad. Cada servicio Python utiliza OpenTelemetry SDK e instrumentación automática para FastAPI, HTTPX y SQLAlchemy, complementada con spans manuales en operaciones de negocio críticas. Los tres tipos de señal se envían mediante OTLP al OpenTelemetry Collector.

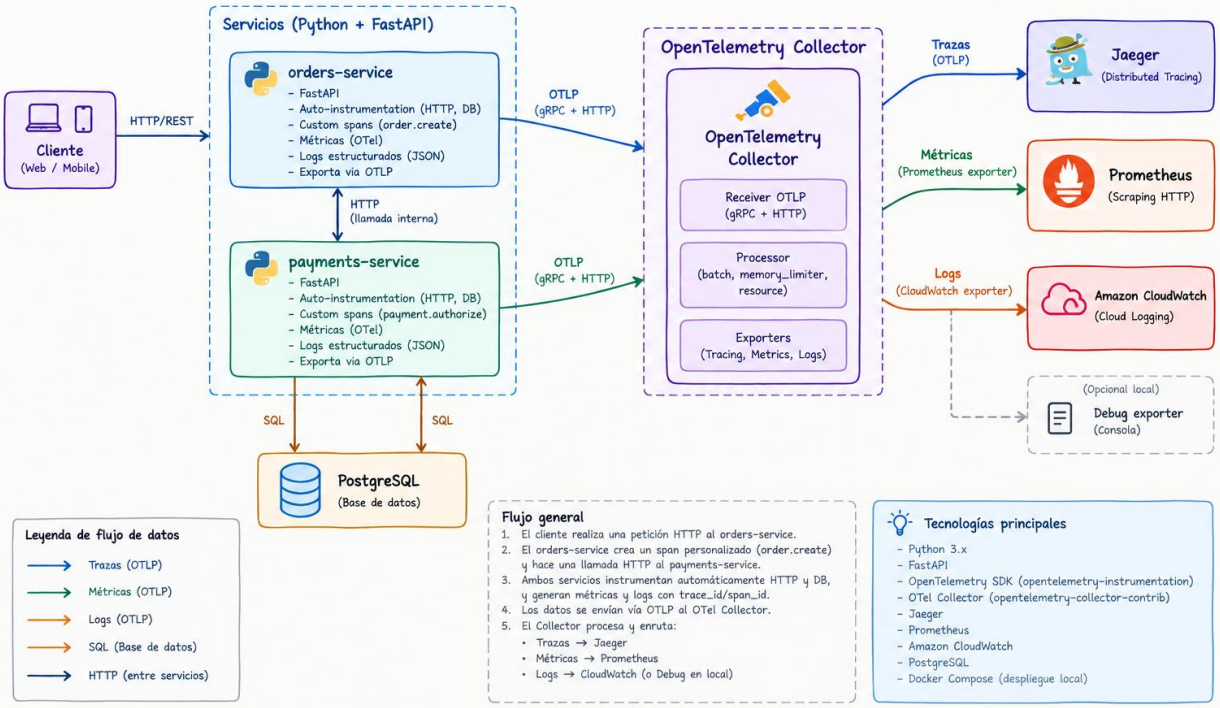
El Collector actúa como gateway de observabilidad: recibe OTLP por gRPC y HTTP, aplica control de memoria, enriquecimiento de recursos y batching, y finalmente enruta trazas hacia Jaeger, métricas hacia un endpoint Prometheus y logs hacia CloudWatch Logs. Esta separación reduce el acoplamiento del código de aplicación con el proveedor de observabilidad y facilita una futura sustitución de backend.

Arquitectura propuesta

El flujo de una operación comienza en orders-service. La creación de una orden genera un span de negocio y una operación SQL instrumentada automáticamente. Posteriormente el servicio llama a payments-service usando HTTPX; el contexto W3C se propaga y permite observar la transacción como una sola traza distribuida.

Arquitectura de Observabilidad con OpenTelemetry

– Servicios Python (Orders & Payments) –



Leyenda de flujo de datos

- Trazas (OTLP)
- Métricas (OTLP)
- Logs (OTLP)
- SQL (Base de datos)
- HTTP (entre servicios)

Flujo general

1. El cliente realiza una petición HTTP al orders-service.
2. El orders-service crea un span personalizado (order.create) y hace una llamada HTTP al payments-service.
3. Ambos servicios instrumentan automáticamente HTTP y DB, y generan métricas y logs con trace_id/span_id.
4. Los datos se envían vía OTLP al OTel Collector.
5. El Collector procesa y enruta:
   - Trazas → Jaeger
   - Métricas → Prometheus
   - Logs → CloudWatch (o Debug en local)

Tecnologías principales

- Python 3.x
- FastAPI
- OpenTelemetry SDK (opentelemetry-instrumentation)
- OTel Collector (opentelemetry-collector-contrib)
- Jaeger
- Prometheus
- Amazon CloudWatch
- PostgreSQL
- Docker Compose (despliegue local)

Decisiones de Diseño

Instrumentación

Decisión	Justificación	Trade-off
SDK configurado en código	Control explícito de providers, exporters y del engine de SQLAlchemy	Más código que mantener
Auto-instrumentación HTTP y BD + spans propios	La auto cubre la infraestructura; la manual agrega contexto de negocio	Los spans propios requieren tocar código
BatchSpanProcessor / BatchLogRecordProcessor	Exportan en segundo plano y por lotes, sin bloquear la petición	Si el proceso muere de golpe, se pierden datos aún no enviados
Métricas cada 10 s	Equilibrio entre tiempo de refresh y costo	El dashboard tiene hasta ~10 s de retraso
Métricas de proceso limitadas a 4	Menos series y menos CPU	Menos detalle del host

Collector y Pipeline

Decisión	Justificación	Trade-off
Apps → Collector (OTLP gRPC)	Desacoplamiento	Un componente más que operar
Orden memory_limiter → resource → batch	Rechazar datos antes de gastar memoria; agrupar al final	
Métricas: OTLP → Prometheus lee del Collector	Un solo punto de lectura; las apps no exponen puertos	El Collector pasa a ser parte del camino de las métricas
resource_to_telemetry_conversion	service_name queda como etiqueta para filtrar en PromQL	Más etiquetas; service.instance.id crea series nuevas en cada reinicio de pod
Telemetría interna en :8888 y panel de la cola del exportador	La cola detecta una caída de backend de inmediato; send_failed tarda ~5 min	

Plataforma GCP

Decisión	Justificación
GKE Autopilot	Jaeger y Prometheus necesitan estar siempre encendidos y tener direcciones internas fijas; Autopilot evita administrar nodos
Cloud Logging	Nativo en GCP
Workload Identity	Sin claves en el cluster

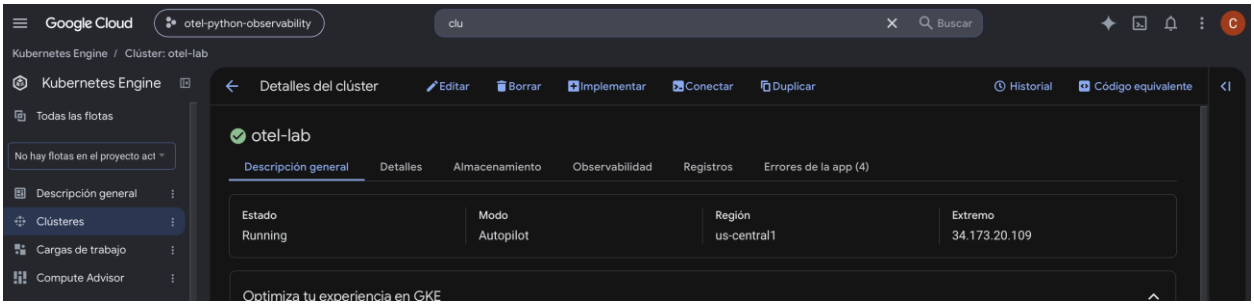
Implementación

SLI y dashboards

- 4 SLIs basados en las golden signals de Google SRE: tráfico, tasa de errores, latencia (p50/p95/p99) y disponibilidad, con un SLO de 99,5%.
- CPU de los servicios y del Collector, para ver el costo de la instrumentación.
- Errores del Collector, para saber si se está perdiendo telemetría.
- Inyección de fallos (PAYMENTS_FAILURE_RATE = 2%), para que los paneles de error tengan datos reales y no queden en cero.

Pruebas

Para la implementación, se creó un proyecto en Google cloud, luego se configuro un contenedor de K8S para más adelante en Google cloud desplegar el cluster correspondiente



Luego para la prueba ejecutamos el script load.sh el cual envía 300 request al servicio de orders para poder obtener data y verla en cada uno de los dashboards de los SLIs

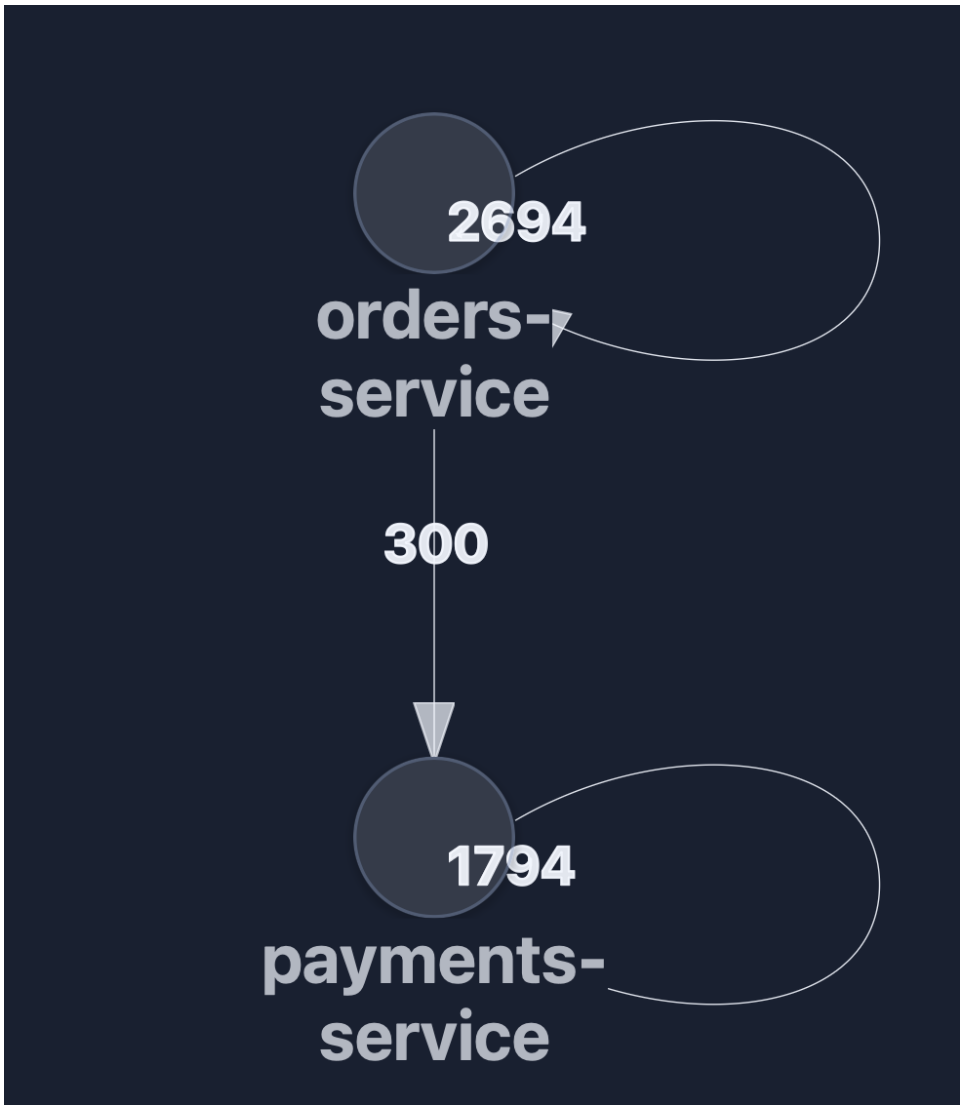
```
carlossalazar@Carloss-MacBook-Air otel-python-observability % scripts/load.sh
HTTP 200: 297 requests
HTTP 500: 3 requests
```

Como resultado de la ejecución tuvimos 3 request con error 500 y 297 request exitosos.

Trazas

Jaeger UI (GCP)

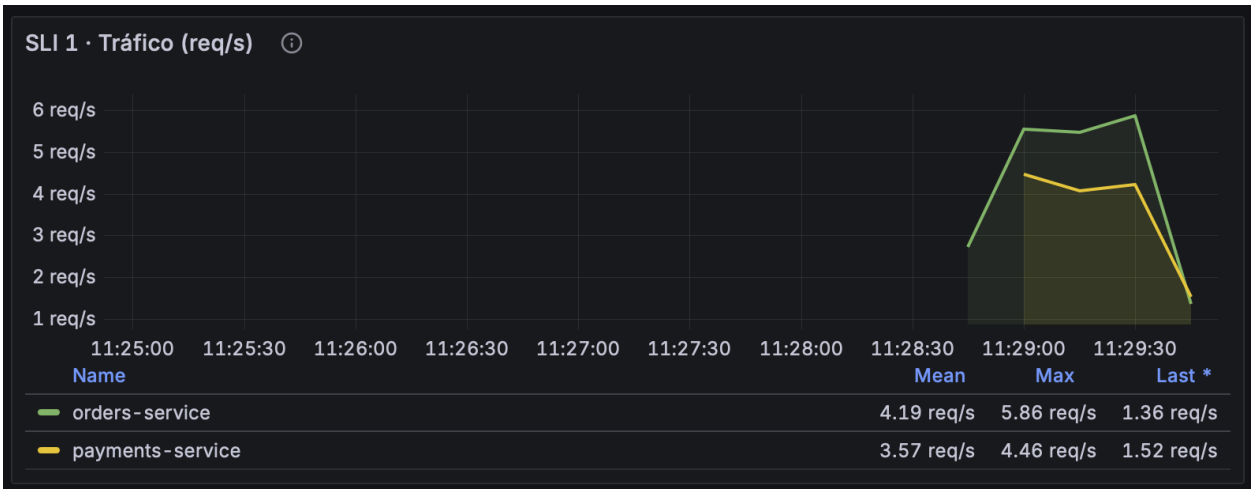
Se valida en Jagger la propagación del contexto entre servicios



Métricas

A continuación se muestra el resultado de la implementación de las métricas de Prometheus + Grafana dashboard con 6 paneles (4 SLIs + CPU + errores OTel Collector)

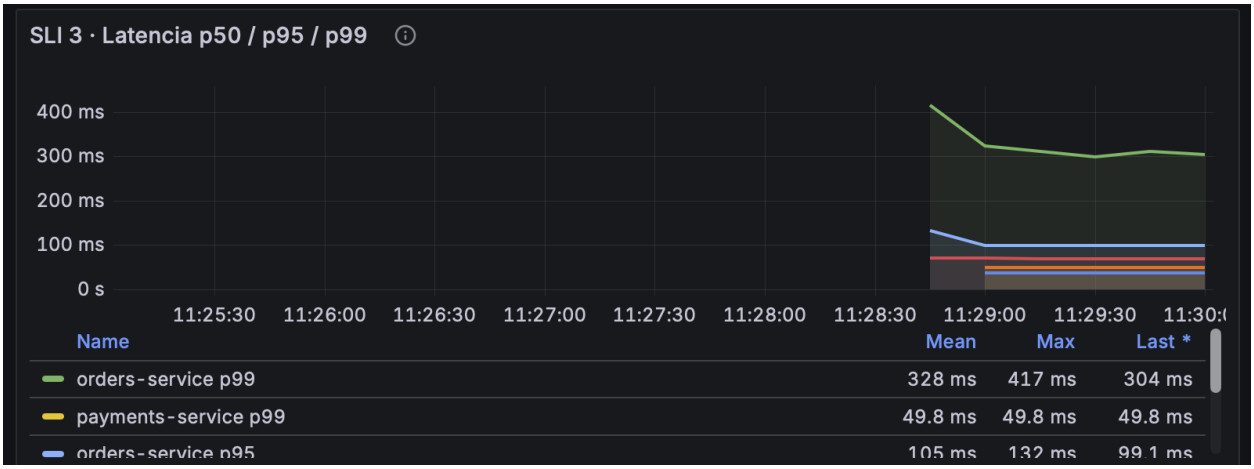
SLI 1 Tráfico



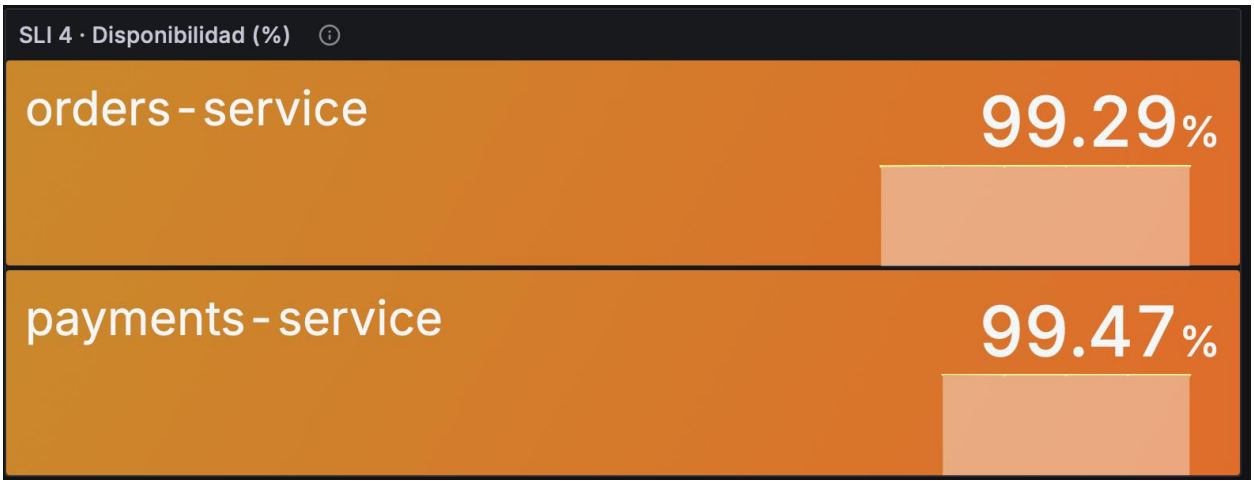
SLI 2 Tasa de errores



SLI 3 Latencia



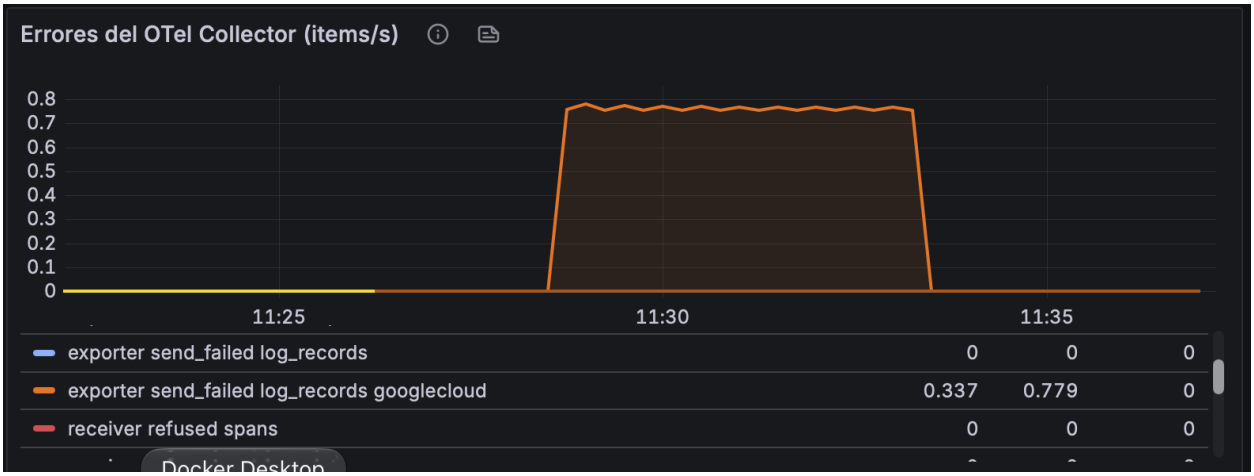
SLI 4 Disponibilidad



CPU



Errores del OTel Collector



Correlación de logs y trazas en Grafana

Responsable Juan José Vélez Álvarez

El aporte conecta los logs estructurados de orders-service y payments-service con su traza distribuida y mide el coste de activar OpenTelemetry. La implementación se verificó en un laboratorio local con Docker Desktop el 11 de octubre de 2026. Las evidencias de GKE de las secciones anteriores corresponden al trabajo del equipo; las nuevas mediciones pertenecen exclusivamente al entorno local.

Diseño del flujo de logs

Ambos servicios envían logs por OTLP/gRPC al Collector. El pipeline aplica memory_limiter, resource y batch, y exporta los registros por OTLP/HTTP al endpoint nativo de Loki. Grafana consulta Loki y utiliza un campo derivado para abrir en Jaeger la traza cuyo identificador aparece en cada registro [1, 2].

El formateador JSON se aplica a la consola y al handler OTLP. Cada log de negocio contiene timestamp en UTC, service, level, message, event, order_id, trace_id y span_id. El identificador de traza se obtiene del contexto activo; no se genera uno nuevo al registrar el mensaje. Los eventos emitidos fuera de una traza llevan identificadores nulos.

Elemento	Implementación
Propagación	W3C traceparent mediante HTTPX y FastAPI
Logs	OTLP/gRPC → Collector → OTLP/HTTP → Loki
Trazas	OTLP/gRPC → Collector → Jaeger
Enlace de logs a trazas	Campo TraceID → datasource UID jaeger
Enlace de trazas a logs	Consulta Loki filtrada por __span.traceId
Etiquetas de Loki	service.name y deployment.environment.name

Decisiones de cardinalidad y alcance

El trace_id se conserva como metadato estructurado y dentro del JSON, sin convertirlo en etiqueta de índice de Loki. Cada petición genera un valor distinto; indexarlo multiplicaría los streams. En Prometheus tampoco se añade trace_id a cada serie. Las métricas son agregados de muchas peticiones; la navegación métrica-traza mediante exemplars queda en el alcance de métricas del equipo y no se presenta como validada en este aporte.

El overlay compose.juan-jose.yaml usa un proyecto y puertos propios. Mantiene la configuración cloud del equipo separada y permite ejecutar la demostración sin credenciales GCP o AWS. La versión local de Loki usa almacenamiento filesystem y no constituye un despliegue de alta disponibilidad.

Evidencia de propagación W3C

La prueba envió una orden con un encabezado traceparent generado para esa petición y comprobó las respuestas de los backends. El trace_id recibido se mantuvo al cruzar la llamada HTTP entre pedidos y pagos. Cada log se vinculó con un span existente de la misma traza y con el mismo order_id [3].

```
Trace ID  d1a1502f734f3700992585ea3fdb7f03
```

```
Order ID  46d220ae-4e65-4d1a-a68f-b1113c578f72
```

Servicio	Evento	Span ID
orders-service	order.persisted	d8640ae86467387f
payments-service	payment.authorized	df2742f185f6fc99
orders-service	order.completed	d8640ae86467387f

La prueba encontró 17 spans y 3 logs de negocio. Verificó operaciones SQL y una relación padre-hijo que cruza de orders-service a payments-service. Los span_id de los servicios son distintos porque identifican operaciones distintas; el trace_id común identifica la transacción completa.

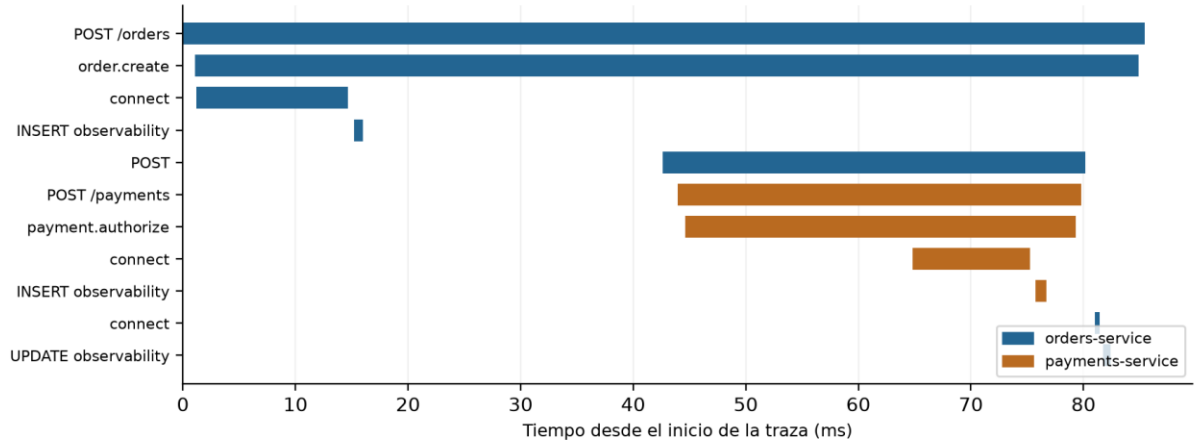


Figura 1. Duración de los spans principales de la traza real, reconstruida desde jaeger.json. Se omiten spans internos de envío/recepción ASGI para facilitar la lectura. Es una visualización de datos exportados, no una captura de la interfaz.

Consulta en Explore

```
{service_name=~"orders-service|payments-service"}
| trace_id="d1a1502f734f3700992585ea3fdb7f03"
```

En Explore, el datasource Loki consulta los registros y el campo derivado TraceID está configurado para abrir Jaeger. La comprobación por API validó la salud de Loki, la expresión de extracción, el UID de destino y la consulta inversa. La evidencia adjunta acredita correlación en los backends y configuración por API; no acredita una prueba visual del enlace en la interfaz. Los JSON originales permiten auditar los identificadores y las relaciones entre spans.

Metodología del benchmark de overhead

Se compararon dos variantes de la misma aplicación e imagen: baseline con OTEL_SDK_DISABLED=true y variante instrumentada con OTEL_SDK_DISABLED=false. La línea base omite la creación de providers, exportadores e instrumentadores; conserva los logs JSON de negocio, las dependencias importadas y las llamadas no-op. Por tanto, el experimento mide el coste incremental de activar la telemetría, no el ahorro de retirar todas sus bibliotecas.

Condición	Valor del experimento
Carga y ventana	50 usuarios concurrentes; 300 s por variante
Calentamiento	30 s antes de cada ventana medida
Interacción	POST /orders; petición HTTP a payments y SQL en PostgreSQL
Tiempo de reflexión	0,5 a 1,5 s; semilla fija en k6
Errores artificiales	PAYMENTS_FAILURE_RATE=0
Trazas	always_on; ambos servicios instrumentados en variante OTel
Recursos por servicio	Límite de 2 CPU y 512 MiB; una instancia
Entorno	Windows 11; Docker Desktop Linux/WSL2; 16 CPU lógicas visibles
Herramientas	k6 2.2.0; Docker Engine 27.4.0; Python 3.13 en contenedores
Orden y repeticiones	Una pareja: baseline seguido de OTel

Antes de cada escenario se reiniciaron los servicios y se vaciaron únicamente las tablas del PostgreSQL del proyecto de benchmark. Se aplicó un enfriamiento de 15 segundos y luego el calentamiento. Los fallos de negocio y HTTP se revisaron con checks y un umbral de error menor al 1 %. Las duraciones de calentamiento se guardaron por separado y no entran en el p99 reportado.

Definición de las medidas

k6 calculó el p99 de http_req_duration: envío, espera y recepción de cada petición, sin el establecimiento de conexión [4]. CPU y memoria se muestrearon externamente con docker stats cada aproximadamente cinco segundos, con el mismo procedimiento en ambos modos. Se sumaron pedidos y pagos en cada instante; luego se calcularon media y pico. El uso de CPU puede superar 100 %, porque 100 % equivale a un núcleo lógico.

La memoria reportada es uso del contenedor según Docker, que en Linux excluye caché; no se interpreta como RSS del proceso [5]. El Collector se registra por separado. Las muestras incluyen el drenaje de las últimas iteraciones, de hasta 15 segundos, y no incluyen el calentamiento. El generador y los backends comparten el mismo equipo, lo cual limita el aislamiento experimental.

Resultados comparativos del experimento local

Medida	Sin OTel	Con OTel	Diferencia
p99 de latencia (ms)	164,65	255,31	90,65
CPU media (% de 1 núcleo)	44,46	61,86	17,41 p.p.
Memoria media (MiB)	323,14	223,36	-99,78
Memoria pico (MiB)	357,98	271,10	-86,88
Memoria total laboratorio (MiB)	653,63	876,63	223,00
Throughput (req/s)	47,66	47,31	-0,35
Errores HTTP (%)	0,00	0,00	0,00 p.p.

El p99 cambió en 90,65 ms, equivalente a 55,06 % respecto a la línea base. El overhead relativo de CPU fue 39,16 % y la memoria media cambió en -99,78 MiB. Estos valores describen una pareja local y no una constante propia de OpenTelemetry.

Cálculo de las diferencias

```
Δp99 = p99 con OTel − p99 sin OTel
CPU overhead (%) = 100 × (CPU con OTel / CPU sin OTel − 1)
Δmemoria = memoria con OTel − memoria sin OTel
```

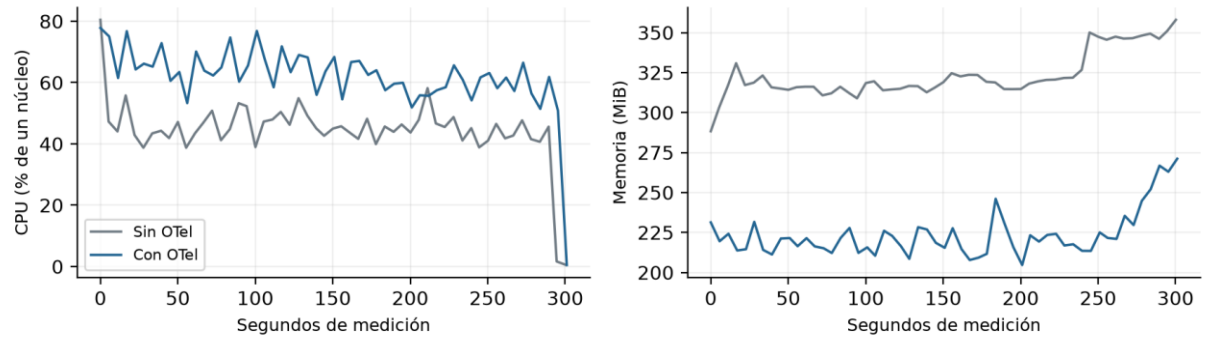


Figura 2. Suma de CPU y memoria de pedidos y pagos durante las ventanas medidas. Las curvas se alinean por tiempo transcurrido de cada escenario. Fuente: resources.csv.

La línea base completó 14,364 peticiones y la variante OTel 14,265. Se obtuvieron 55 y 55 muestras de recursos, respectivamente. El código de salida de k6 y el muestreo determinaron un estado de validación PASS. El Collector consumió en promedio 0,09 % / 0,98 % de CPU y 55,13 / 105,40 MiB, en baseline y OTel.

Interpretación y límites de la evaluación

La instrumentación añade creación y procesamiento de spans, enriquecimiento de logs, actualización de métricas, colas de exportación y transporte OTLP. BatchSpanProcessor, BatchLogRecordProcessor y la exportación periódica amortizan el envío, aunque mantienen trabajo en segundo plano. En esta prueba el coste se mide con todos esos componentes activos y muestreo de trazas del 100 %. Un muestreo menor cambiaría el resultado y la cobertura de correlación.

El indicador de CPU es relativo a la línea base: no debe confundirse con puntos porcentuales de utilización. La memoria de las dos aplicaciones cambió en -99,78 MiB, mientras la suma de los ocho contenedores del laboratorio cambió en 223,00 MiB. Esta segunda medida incluye PostgreSQL, Collector, Loki, Jaeger, Prometheus y Grafana, y depende de la retención acumulada de telemetría. Separar ambas fronteras evita confundir menor memoria de una aplicación con menor coste total de observabilidad.

Una sola pareja no permite atribuir toda diferencia exclusivamente al SDK ni establecer un intervalo de confianza. El calentamiento, el orden baseline/OTel, las cachés, la frecuencia dinámica del procesador y la contención de Windows/WSL2 pueden afectar las colas de latencia. Un valor negativo no demostraría que OTel acelera la aplicación; sería necesario repetir y estudiar la dispersión. El p99 también depende del número de peticiones completadas.

Repetición y comparación en cloud

El comando python benchmark/run.py --repetitions 3 ejecuta parejas alternando el orden A/B y B/A. El analizador conserva cada corrida y calcula la mediana de sus medidas; la mediana de varios p99 se etiqueta como tal y no como un percentil global. Para aumentar la confianza se recomienda separar el generador de carga del sistema observado y mantener fijas las réplicas y los recursos.

El repositorio conserva la configuración GKE del equipo. La integración cloud de logs requiere agregar otlphttp/loki al pipeline existente sin retirar googlecloud en GKE ni awscloudwatchlogs en ECS, y aprovisionar el datasource Loki en Grafana. Se debe usar conectividad privada y almacenamiento persistente. Este aporte no acredita un nuevo despliegue ECS ni un benchmark ejecutado en GKE; ambas validaciones deberán realizarse y reportarse por separado.

Relación con el indicador de desempeño

El trace_id permite pasar de un evento de negocio al tramo HTTP o SQL asociado, lo que facilita localizar dónde se produce una falla. La prueba demuestra la unión técnica entre señales, pero no mide una reducción de MTTR ni un aumento de resiliencia: para cuantificar esos beneficios sería necesario diseñar incidentes controlados y medir tiempos de detección y resolución. El benchmark permite evaluar el coste de mantener esa capacidad de diagnóstico.

Evidencias conservadas

La carpeta de resultados contiene los agregados originales de k6, los CSV temporales de recursos, ventanas UTC, versiones, diagnósticos del Collector, comparacion.csv/json/md y las respuestas completas de Loki y Jaeger. El repositorio incluye un script de repetición y pruebas aisladas de los dos modos del SDK. Las capturas originales del despliegue GKE permanecen en las secciones previas del documento.

Reproducción y referencias del aporte

Pasos para revisar la entrega

1. Abra la carpeta otel-python-observability y revise README-JUAN-JOSE.md. El archivo compose.juan-jose.yaml define los puertos y el proyecto aislado del experimento.

2. Ejecute python benchmark/run.py con Docker Desktop activo y k6 instalado. El script verifica la correlación, mide ambos escenarios y restaura los servicios con OTel activado.

3. Revise benchmark/results/LATEST.txt para identificar la última corrida. Compare summary.json y resources.csv con los cálculos de comparacion.json.

4. Abra Grafana en localhost:13000, elija Explore y Loki, y use la consulta con el trace_id de verification.json. Expanda un log y abra Ver traza en Jaeger.

5. Conserve una captura de los logs con el trace_id y otra de la traza que muestre pedidos, pagos y SQL. Identifique el entorno y el momento de captura.

6. Para detener el laboratorio conservando los volúmenes, ejecute docker compose -p otel-juan-jose -f docker-compose.yml -f compose.juan-jose.yaml stop.

Archivos principales modificados o incorporados

Archivo	Propósito
services/*/app/otel.py	Modo no-op y JSON homogéneo en consola y OTLP
loki/loki.yaml	Ingesta OTLP y metadatos estructurados
grafana/juan-jose/datasources.yaml	Enlaces Loki → Jaeger y Jaeger → Loki
otel-collector-juan-jose.yaml	Envío local de logs a Loki
benchmark/load.js y run.py	Carga k6 y recolección externa
benchmark/analyze.py	Cálculo y tabla comparativa
benchmark/verify_correlation.py	Verificación W3C y evidencias de backend

Referencias

[1] Grafana Labs. Ingesting logs to Loki using OpenTelemetry Collector. https://grafana.com/docs/loki/latest/send-data/otel/

[2] Grafana Labs. Configure the Loki data source. https://grafana.com/docs/grafana/latest/datasources/loki/configure/

[3] W3C. Trace Context. https://www.w3.org/TR/trace-context/

[4] Grafana Labs. k6 built-in metrics. https://grafana.com/docs/k6/latest/using-k6/metrics/reference/

[5] Docker. docker container stats. https://docs.docker.com/reference/cli/docker/container/stats/

Código base del equipo: https://github.com/juansebastian-br/otel-python-observability, commit e1d234c. Documentación técnica consultada el 11 de octubre de 2026.